MongoDB Queries



Máster Big Data y Business Analytics — 7 de enero de 2019

Introducción

El objetivo del presente trabajo consiste en elegir una colección de datos, incorporarlos a MongoDB y hacer un análisis de ésta, desarrollando las consultas necesarias para ello.Para dicho análisis he elegido un listado de todos los *Pokémon* de primera generación recogidos en la *Pokédex* del videojuego *Pokémon GO*, así como sus estadísticas.

Este archivo ha sido extraído del repositorio público del usuario **Gianluca Bonifazi** en *Github*[1].

No obstante, este fichero ha sido tratado para facilitar las distintas *queries*, ya que originalmente se encuentran todos los Pokémon en un solo fichero, dentro de un *array* llamado "pokemon". Además, el fichero original tiene un campo *id* para cada Pokémon, por lo que fue eliminado para que no hubiese conflicto con el *id* que asigna automáticamente MongoDB. Esta operación se realizó mediante el siguiente comando:

```
db.Pokemon.updateMany({},{$unset: {'id': ''}})
```

Los campos que componen cada Pokémon son:

- num: Número del Pokémon en la Pokédex oficial.
- name: Nombre del Pokémon.
- img: URL a una imagen del Pokémon.
- type: Tipo de Pokémon.
- height: Altura del Pokémon.
- weight: Peso de cadaPokémon.
- candy: Tipo de caramelo utilizado para evolucionar el Pokémon o que se recibe al transferirlo.
- candy_count: Número de caramelos necesarios para evolucionar.
- egg: Número de kilómetros necesarios para eclosionar un huevo.
- spawn_chance: Porcentaje de la probabilidad de aparición.
- avg_spawns: Número de apariciones de este Pokémon cada 10.000 apariciones.
- spawn_time: Momento con mayor número de apariciones activas para una determinada zona. Este número es el mismo para todas las zonas horarias y están expresados en hora local.

[1] https://github.com/Biuni/PokemonGO-Pokedex

- multipliers: Multiplicador del Poder de Combate (CP) para calcular el CP tras evolucionar el Pokémon.
- weakness: Tipos de Pokémon a los que el Pokémon es débil.
- next_evolution: Número y número de las evoluciones siguientes del Pokémon.
- prev_evolution: Número y número de las evoluciones previas del Pokémon.

1. Carga del fichero json

Para cargar el fichero se hizo uso de la herramienta *mongoimport* de MongoDB mediante la consola de Windows. El comando fue el recogido en la Figura 1.

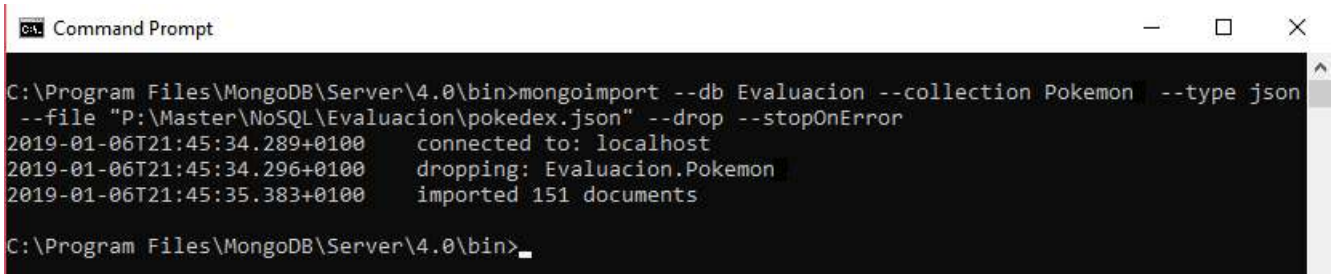


Figura 1: Comando para la carga del archivo JSON en MongoDB

2. Queries

Una vez insertado el documento en MongoDB se procede a examinar su estructura, la información que contiene. Para ello nos podemos ayudar de la función de MongoDB Compass llamada *Schema*, que nos ayuda a determinar toda la información recogida en los distintos campos del documento y de qué forma están almacenados.

Tras un análisis preliminar se pueden elaborar las distintas queries destinadas a obtener la información requerida. Estas queries son las siguientes:

- En primer lugar, saber cuántos Pokémon de primera generación están recogidos en la Pokédex.

```
db.Pokemon.find().count()
```

- Del total de Pokémon, calcular cuántos no tienen ninguna evolución.

```
db.Pokemon.find({'next_evolution': {$exists: false}}).count()
```

- Los Pokémon se categorizan según el tipo que sean, pudiendo ser de varios tipos al mismo tiempo. Por ello es interesante calcular cuántos Pokémon existen de cada tipo. Se presentan ordenados de mayor a menor.

```
db.Pokemon.aggregate([{$unwind: '$type'},
                      {$group:
                                {_id: '$type', Total: {$sum: 1}
                                }
                      },
                      {$sort: {Total: -1}
                      }
                     ])
```

- Según el tipo que sea cada Pokémon, hará que sean débiles frente a otros tipos a la hora de enfrentarse. Por ejemplo,un Pokémon tipo *Grass* será débil frente a un Pokémon tipo *Fire*. Por ello, a continuación se extraen las 5 debilidades más comunes.

```
db.Pokemon.aggregate([{$unwind: '$weaknesses'},
                      {$group:
                                {_id: '$weaknesses', Total: {$sum: 1}
                                }
                      },
                      {$sort: {Total: -1}
                      },
                      {$limit: 5}])
```

- Dado que tenemos un campo con la altura, es interesante saber cuál es el más alto.

```
db.Pokemon.find({},{_id: 0, 'name': 1, 'height': 1}).sort({height: -1})
                .limit(1)
```

- Asímismo, cuál es el más pequeño.

```
db.Pokemon.find({},{_id: 0,'name': 1, 'height': 1}).sort({height: 1})
                .limit(1)
```

- Al igual que con la altura, también está recogido el peso. Por ello, podemos obtener el Pokémon más pesado.

```
db.Pokemon.find({},{_id: 0,'name': 1, 'weight': 1}).sort({weight: -1})
                .limit(1)
```

- Y también el Pokémon más ligero.

```
db.Pokemon.find({},{_id: 0,'name': 1, 'weight': 1}).sort({weight: 1})
                .limit(1)
```

- Una de las formas que el jugador tiene disponible para obtener los Pokémon es mediante huevos, que una vez recorrida cierta distancia (2km, 5km o 10km), eclosionan. Mediante esta *query* obtenemos el número de Pokémon que es posible obtener, dentro de esta modalidad, según los kilómetros necesarios y, por último, los que no son posibles obtener mediante la eclosión de huevos.

```
db.Pokemon.aggregate([{$group:
                                {_id: '$egg', Total: {$sum: 1}}},
                      {$sort: {Total: -1}
                      }])
```

- Mediante la siguiente query se obtiene el número de Pokémon existentes según el caramelo y, también, cuántos no tienen ninguno.

```
db.Pokemon.aggregate([{$group:
                              {_id: '$candy', Total: {$sum: 1}
                              }
                      },
                      {$sort: {Total: -1}
                      }])
```

- También es interesante obtener la media de caramelos necesarios para evolucionar los Pokémon según su tipo.

```
db.Pokemon.aggregate([{$unwind: '$type'},
                      {$group:
                               {_id: '$type',
                               Media: {$avg: '$candy_count'}
                               }
                      },
                      {$sort: {Media: -1}
                      }])
```

- No todos los Pokémon tienen multiplicador, la siguiente consulta nos permite saber cuántos son. Mediante ésta, se obtienen que son 81 Pokémon, lo cual coincide con el número de Pokémon sin evolución. Dado que el multiplicador, según su definición, sirve para calcular el CP resultante al evolucionar el Pokémon, el resultado es correcto.

```
db.Pokemon.find({multipliers: null}).count()
```

- Dado que cada Pokémon tiene un tiempo distinto *spawn*, es interesante saber cuál es este tiempo, de media, según los tipos de Pokémon.

```
db.Pokemon.aggregate([{$unwind: '$type'},
                      {$group:
                               {_id: '$type',
                               Media: {$avg: '$avg_spawns'}
                               }
                      },
                      {$sort: {Media: -1}
                      }])
```

- Por último, calcular la longitud media del nombre de los Pokémon.

```
db.Pokemon.aggregate([{$project:
                               {_id: '$name',
                               'length': {$strLenCP: '$name'}
                               }
                      },
                      {$group:
                               {_id: null,
                               Average_characters: {$avg: '$length'}
                               }
                      }])
```

3. Conclusiones

MongoDB nos proporciona una forma fácil y rápida de almacenamiento, además de ser completamente flexible en cuanto a la estructura de los datos se refiere. La sintaxis del lenguaje para elaborar las consultas es bastante sencillo y directo, por lo que se pueden organizar rápida y eficientemente.

La cantidad de información disponible en los recursos oficiales de MongoDB son de inestimable ayuda a la hora de buscar información y conocer las funciones necesarias para obtener las consultas deseadas, así como su estructura y funcionamiento. También es de gran ayuda la aplicación MongoDB Compass, con funciones tan útiles como la función *Schema*, arriba mencionada. Esta funcionalidad analiza la base de datos y nos muestra su estructura y los contenidos de los distintos campos organizados visualmente. De esta forma, se puede realizar un análisis preliminar del fichero y empezar a dibujar qué consultas son interesantes realizar y cómo hacerlas.

Si bien para consultas más básicas con el operador *find*, estableciendo los filtros adecuados para obtener la información buscada, es suficiente, para las consultas más complejas he utilizado la función *aggregate*. De esta forma, se pueden encadenar distintos filtros y operadores hasta obtener el dato buscado. Otra función interesante, dado que había campos que erran *arrays*, es la función *unwind*, que permite extraer la información del *array* y así trabajar con esta información cómodamente.

Por último, destacar que me ha sorprendido la facilidad y la flexibilidad con que se trabaja con MongoDB, una vez trabajas con esta base de datos entiendes su gran crecimiento y popularidad.